

Step 1 — Connect to One MCP Server and List Its Tools — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume this tiny server, saved as `example_server.py` in the same folder (the minimal shape from Doc06's Core Concepts):

```
# example_server.py
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("example-server")

@mcp.tool()
def get_weather(city: str, country_code: str) -> str:
    """Get the current weather for a specific city."""
    return f"18C, cloudy, in {city}, {country_code}"

@mcp.tool()
def add_numbers(a: float, b: float) -> float:
    """Add two numbers together."""
    return a + b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Basic Version

Approach 1 — the direct way

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
```

```
async def main():
    server_params = StdioServerParameters(command="python", args=["example_server"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            for tool in tools_result.tools:
                print(tool.name, "-", tool.description)

asyncio.run(main())
```

Expected output:

```
get_weather - Get the current weather for a specific city.
add_numbers - Add two numbers together.
```

This works correctly for the happy path. It prints directly inside `main()` instead of returning anything, has no type hints, and does nothing special if the command is wrong or the tool list is empty — all fine for a first working version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a reusable function that returns the tools instead of printing them

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp.types import Tool

async def list_server_tools(command: str, args: list[str]) -> list[Tool]:
    """Connect to one MCP server over stdio and return its discovered tools."""
    server_params = StdioServerParameters(command=command, args=args)
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            return tools_result.tools

async def main() -> None:
    tools = await list_server_tools("python", ["example_server.py"])
    for tool in tools:
        print(f"{tool.name}: {tool.description}")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
get_weather: Get the current weather for a specific city.
add_numbers: Add two numbers together.
```

Difference from Basic: the connection logic is now its own typed function that *returns* the tools instead of printing them inline — this matters because Step 2 needs the tool list as a value it can hand to the model, not just something printed to a terminal. `mcp.types.Tool` gives real type hints for what comes back, instead of an untyped object.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — prints the full input schema, and handles the empty-tools case on purpose

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp.types import Tool

async def list_server_tools(command: str, args: list[str]) -> list[Tool]:
    server_params = StdioServerParameters(command=command, args=args)
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            return tools_result.tools

def print_tools(tools: list[Tool]) -> None:
    if not tools:
        print("Connected fine -- this server just has no tools registered.")
        return
    for tool in tools:
        print(f"- {tool.name}: {tool.description}")
        print(f"  input schema: {tool.inputSchema}")

async def main() -> None:
    tools = await list_server_tools("python", ["example_server.py"])
    print_tools(tools)

if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

- `get_weather`: Get the current weather for a specific city.
input schema: `{'properties': {'city': {'title': 'City', 'type': 'string'}, 'cou`
- `add_numbers`: Add two numbers together.
input schema: `{'properties': {'a': {'title': 'A', 'type': 'number'}, 'b': {'tit`

Notice `inputSchema` is already plain JSON Schema (`type` , `properties` , `required`) — this is exactly the shape Step 2 hands almost unchanged into OpenAI's `parameters` field.

Approach 2 — the deliberately broken command, named and handled

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp.types import Tool

async def list_server_tools(command: str, args: list[str]) -> list[Tool]:
    server_params = StdioServerParameters(command=command, args=args)
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools_result = await session.list_tools()
            return tools_result.tools

async def main() -> None:
    print("== working server ==")
    tools = await list_server_tools("python", ["example_server.py"])
    for tool in tools:
        print(f"- {tool.name}: {tool.description}")

    print("\n== deliberately wrong command ==")
    try:
        await list_server_tools("python", ["this_file_does_not_exist.py"])
    except Exception as e:
        print(f"Failed as expected: {type(e).__name__}: {e}")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
== working server ==  
- get_weather: Get the current weather for a specific city.  
- add_numbers: Add two numbers together.  
  
== deliberately wrong command ==  
Failed as expected: ProcessLookupError: ...
```

The exact exception type and message vary a little by OS and Python version -- the point isn't memorizing one exact string, it's confirming *some* clear, catchable error surfaces here, instead of the process hanging with no output at all. Run this on your own machine and read whatever it actually prints; that's the real failure signature you'll be handling for real in Step 4.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `list_server_tools` only ever gets called against a server that already works. Approach 1 adds the other correctness piece Intermediate skipped: printing the full schema (which Step 2 actually needs) and treating an empty tool list as a valid, non-error result. Approach 2 adds the failure case Hint 1 warned about -- proving, with your own eyes, exactly what a bad server command actually does on your machine, before Step 4 has to catch and handle it gracefully instead of just observing it.

Which one should you actually write? Intermediate's Approach 1 is the right shape to carry forward into Step 2 -- a small, typed, reusable `list_server_tools()` that returns data instead of printing it. Do add Advanced Approach 1's full schema printing while you're building Step 1, even briefly, so you've actually looked at a real `inputSchema` with your own eyes before Step 2 asks you to convert it. Advanced Approach 2's broken-command test is worth running once, by hand, right now -- you don't need to keep it as permanent code in this step, but seeing the real failure mode now is exactly what makes Step 4's fallback logic feel like solving a problem you've actually met, instead of one you're handling in the abstract.